

BrainWash: A Poisoning Attack to Forget in Continual Learning

论文分析笔记

2024 CVPR

1 问题背景

BrainWash 的目标不是让当前任务分类错误，而是让连续学习模型在学习当前任务 T 之后遗忘旧任务 $1, \dots, T-1$ 。论文考虑的核心威胁模型是：攻击者能访问已经学完旧任务的模型，以及当前任务的 clean training data；攻击者不能访问旧任务真实训练数据，也不知道 victim 具体使用哪种 CL 方法和超参数。

因此，攻击分为两步：

1. 对 victim model 做 model inversion，合成旧任务代理数据集 $\hat{D}_{1:T-1}$ ；
2. 在当前任务数据 D_T 上优化小扰动 δ ，让 victim 用 $x_T^i + \delta^i$ 训练后在旧任务代理数据上损失变大。

2 “the logits in the t' th head” 中的 head 是什么

论文把 CL 模型分成共享 backbone 和任务特定分类头：

$$z_t = h_t(f(x; \theta); \psi_t) \in \mathbb{R}^{K_t}.$$

其中：

- $f(x; \theta)$ ：共享 backbone，用于提取输入特征；
- $h_t(\cdot; \psi_t)$ ：第 t 个任务的 classification head；
- z_t ：第 t 个 head 输出的 logits，也就是 softmax 之前的类别分数；
- K_t ：第 t 个任务的类别数。

所以 “the logits in the t' th head” 指的是：把样本输入共享 backbone 后，再送入第 t 个任务专用分类器，该分类器输出的 K_t 维 logits。攻击者可以通过 logits 的维度推断第 t 个任务有多少类。

2.1 为什么会有多个 head

多 head 是 task-incremental continual learning 中很常见的设定。如果每个任务的标签空间不同，并且测试时知道 task id，就可以给每个任务一个单独的分类器：

$$\text{model} = f(\cdot; \theta) + \{h_1(\cdot; \psi_1), h_2(\cdot; \psi_2), \dots, h_T(\cdot; \psi_T)\}.$$

例如，10-split CIFAR-100 中每个任务包含 10 个类别。task 1 使用 h_1 ，task 2 使用 h_2 ，以此类推。这样做的好处是不同任务的分类边界可以部分隔离；共享部分主要负责通用表示学习，而每个 head 只负责自己的任务标签空间。

如果你们在 BrainUICL 中没有遇到 head 概念，通常说明你们使用的是 single-head 设置。single-head 中所有任务共享同一个输出层：

$$z = h(f(x; \theta); \psi) \in \mathbb{R}^{K_{\text{all}}}.$$

这种设置更常见于 class-incremental learning，或者任务之间共享固定标签空间的场景。BrainWash 主实验针对 regularization-based multi-head CL，但论文在 replay-based ER 和 ER-ACE 实验中也评估了 single-head 网络。

3 BrainWash 中的 Model Inversion

3.1 MI 的目的

BrainWash 假设攻击者没有旧任务真实数据。但是攻击者要优化“让模型忘掉旧任务”，就必须有某种数据来估计旧任务损失。因此，作者先通过 model inversion 合成旧任务代理数据：

$$\hat{D}_t = \{(\hat{x}_t^i, \hat{y}_t^i)\}_{i=1}^M, \quad t \in \{1, \dots, T-1\}.$$

$\hat{D}_t$ 不是 victim 的真实训练集，而是攻击者通过已训练模型反推出的 proxy dataset。后续的 poisoning 优化会用这些代理数据衡量旧任务遗忘程度。

3.2 “randomly sampled target one-hot labels” 的含义

论文中的“formulate the MI for a set of randomly sampled target one-hot labels”意思是：攻击者先为第 t 个任务随机采样一批目标标签：

$$\{\hat{y}_t^i\}_{i=1}^M, \quad \hat{y}_t^i \in \mathcal{Y}_t.$$

如果第 t 个任务有 K_t 类，那么每个 $\hat{y}_t^i$ 是一个 one-hot 向量。例如目标类别为第 3 类时：

$$\hat{y} = [0, 0, 1, 0, \dots, 0]^\top.$$

然后攻击者随机初始化输入 x_i ，固定 victim model 参数，只优化输入本身，使模型在第 t 个 head 上把 x_i 预测成 $\hat{y}_t^i$ 对应的类别。

3.3 MI 样本来自哪里

MI 过程中的样本不是旧任务真实训练样本，也不是 victim 训练过程中的 mini-batch。它们来自攻击者的输入优化过程：

1. 攻击者拿到已学完旧任务的模型参数 $\theta_{1:T-1}^*$ 和旧任务 head 参数 ψ_t^* ；
2. 攻击者随机采样目标 one-hot 标签 $\hat{y}_t^i$ ；
3. 攻击者随机初始化输入 x_i ，例如从噪声开始；
4. 固定模型参数，只对 x_i 做梯度下降；
5. 得到合成样本 $\hat{x}_t^i$ ，构成代理数据集 $\hat{D}_t$ 。

因此，MI 样本由攻击者合成，不是模型训练过程中直接保存或提供的旧样本。

3.4 MI 公式

论文采用类似 DeepInversion 的 model inversion 方法。对第 t 个旧任务，MI 的优化目标为：

$$\begin{aligned} \{\hat{x}_t^i\}_{i=1}^M = \arg \min_{\{x_i \in \mathcal{X}\}_{i=1}^M} & \sum_{i=1}^M \mathcal{L}(x_i, \hat{y}_t^i, \theta_{1:T-1}^*, \psi_t^*) \\ & + \sum_{i=1}^M R_{\text{prior}}(x_i) + \alpha_f R_{\text{feat}}(\{x_i\}_{i=1}^M, \theta_{1:T-1}^*). \end{aligned}$$

分类损失可以写成：

$$\mathcal{L}(x_i, \hat{y}_t^i, \theta, \psi_t) = \text{CE}\left(\text{softmax}(h_t(f(x_i; \theta); \psi_t)), \hat{y}_t^i\right).$$

图像先验项为：

$$R_{\text{prior}}(x) = \alpha_{\text{TV}} R_{\text{TV}}(x) + \alpha_{\ell_2} R_{\ell_2}(x).$$

其中 R_{TV} 是 total variation 正则，用来鼓励图像局部平滑； R_{ℓ_2} 是输入范数正则，用来控制像素幅度。

特征统计正则项为：

$$R_{\text{feat}}(\{x_i\}_{i=1}^M, \theta_{1:T-1}^*) = \sum_l \|\mu_l(\{x_i\}_{i=1}^M) - m_l\|_2^2 + \sum_l \|\sigma_l^2(\{x_i\}_{i=1}^M) - v_l\|_2^2.$$

这里 m_l 和 v_l 是第 l 个 BatchNorm 层保存的 running mean 和 running variance； μ_l 和 σ_l^2 是当前合成样本在该层产生的 batch 特征均值和方差。

3.5 MI 公式的原理和最终目标

MI 的优化变量是输入样本 $\{x_i\}$ ，不是模型参数。模型参数 $\theta_{1:T-1}^*$ 和 ψ_t^* 在 MI 中保持固定。

MI 最终优化的是三件事：

- 分类匹配：让合成样本被第 t 个 head 判成指定 one-hot 标签；
- 自然图像先验：让合成样本不要退化成为难以解释的高频噪声；
- 特征统计匹配：让合成样本在 BatchNorm 层中的统计接近模型训练时见过的数据分布。

最终输出是旧任务代理数据集 $\hat{D}_{1:T-1}$ 。这些数据主要用于攻击者内部优化 poisoning noise，不是直接提供给 victim 训练。

4 BrainWash 的 poisoning 优化目标

4.1 Reckless attacker

BrainWash 的主要优化目标是污染当前任务 T 的训练数据：

$$x_T^i \rightarrow x_T^i + \delta^i, \quad \|\delta^i\|_\infty < \epsilon.$$

当 victim 用这些 poisoned samples 训练后，模型在旧任务代理数据上的损失应尽可能大。论文把 reckless attacker 写成双层优化：

$$\{\delta_T^i\}_{i=1}^{N_T} = \arg \max_{\{\delta^i\}_{i=1}^{N_T}} \sum_{t=1}^{T-1} \sum_{j=1}^M \mathcal{L}(\hat{x}_t^j, \hat{y}_t^j, \tilde{\theta}(\delta), \psi_t^*).$$

subject to:

$$\tilde{\theta}(\delta), \tilde{\psi}_T(\delta) = \arg \min_{\theta, \psi_T} \sum_{i=1}^{N_T} \mathcal{L}(x_T^i + \delta^i, y_T^i, \theta, \psi_T),$$

$$\|\delta^i\|_\infty < \epsilon, \quad \forall i.$$

内层优化表示：攻击者模拟 victim 用 poisoned current-task data 训练模型。外层优化表示：选择扰动 δ ，使训练后的模型在旧任务代理集上的 loss 最大。论文中攻击者在内层只做普通 fine-tuning，并不假设知道 victim 的具体 CL 算法。

4.2 Cautious attacker

reckless attacker 只追求旧任务遗忘，可能导致当前任务 T 也学不好。如果 victim 会监控当前任务 clean validation accuracy，攻击者需要同时保持当前任务表现。因此 cautious attacker 在外层目标中加入当前任务 clean loss：

$$\{\delta_T^i\}_{i=1}^{N_T} = \arg \max_{\{\delta^i\}_{i=1}^{N_T}} \sum_{t=1}^{T-1} \sum_{j=1}^M \mathcal{L}(\hat{x}_t^j, \hat{y}_t^j, \tilde{\theta}(\delta), \psi_t^*) - \eta \sum_{i=1}^{N_T} \mathcal{L}(x_T^i, y_T^i, \tilde{\theta}(\delta), \tilde{\psi}_T(\delta)).$$

其中 $\eta > 0$ 控制“诱导遗忘”和“保持当前任务准确率”之间的权衡。

4.3 为什么要先 MI 获取代理数据集

因为攻击者没有旧任务真实数据。如果没有 $\hat{D}_{1:T-1}$ ，攻击者无法计算：

$$\sum_{t=1}^{T-1} \sum_{j=1}^M \mathcal{L}(\hat{x}_t^j, \hat{y}_t^j, \tilde{\theta}(\delta), \psi_t^*),$$

也就无法知道某个扰动 δ 是否真的会导致旧任务遗忘。

所以 MI 数据集的作用是作为旧任务数据的替代品，用于外层 poisoning objective。攻击者最终提供给 victim 的是 poisoned current-task data：

$$\tilde{D}_T = \{(x_T^i + \delta^i, y_T^i)\}_{i=1}^{N_T},$$

而不是把 MI 合成的旧任务样本直接交给 victim。

5 攻击影响的是 buffer 还是模型参数

BrainWash 的核心影响对象是模型参数，尤其是共享 backbone 参数：

$$\theta_{1:T-1}^* \longrightarrow \tilde{\theta}(\delta).$$

对 regularization-based CL 方法，例如 EWC、MAS、RWALK、AFEC、ANCL，通常没有 replay buffer，攻击者通过污染当前任务训练数据，使 victim 的参数更新方向偏向遗忘旧任务。

对 replay-based CL 方法 ER 和 ER-ACE，论文额外讨论了 memory access threat model。如果攻击者有 memory read access，可以用 buffer 里的旧样本辅助攻击；如果没有，就仍然依靠 MI 代理数据。论文没有考虑直接 write access to memory buffer，因为那会是更简单且不同的威胁模型。因此，BrainWash 的主要机制不是篡改 buffer，而是通过 poisoned current-task data 改变训练后的模型参数。

6 论文涉及的 CL 方法

6.1 Regularization-based CL 的统一形式

论文把 regularization-based CL 的高层形式写成：

$$\theta_{1:T}^* = \arg \min_{\theta} \sum_{i=1}^{N_T} \mathcal{L}(x_T^i, y_T^i, \theta) + \lambda R_{\text{CL}}(\theta, \theta_{1:T-1}^*).$$

第一项让模型学习当前任务；第二项 R_{CL} 限制模型不要过度偏离旧任务重要参数； λ 控制 stability-plasticity trade-off。

6.2 EWC: Elastic Weight Consolidation

EWC 使用 Fisher Information 估计参数对旧任务的重要性。新任务训练时惩罚重要参数偏离旧值：

$$\mathcal{L}_T(\theta) + \frac{\lambda}{2} \sum_j F_j (\theta_j - \theta_j^*)^2.$$

其中 F_j 越大，表示参数 θ_j 对旧任务越重要，训练新任务时越不应该大幅修改。

6.3 MAS: Memory Aware Synapses

MAS 也保护重要参数，但重要性来自模型输出对参数的敏感度，不强依赖 ground-truth label。一个常见表达为：

$$\Omega_j = \frac{1}{N} \sum_x \left\| \frac{\partial \|F(x; \theta)\|_2^2}{\partial \theta_j} \right\|.$$

新任务目标为：

$$\mathcal{L}_T(\theta) + \lambda \sum_j \Omega_j (\theta_j - \theta_j^*)^2.$$

如果某个参数变化会显著改变模型输出，它就会被认为重要。

6.4 RWALK: Riemannian Walk

RWALK 可以理解为 EWC 和路径积分思想的结合。它不仅考虑任务结束点附近的参数重要性，也考虑训练轨迹中参数变化对 loss 的贡献。其目标是在避免 catastrophic forgetting 的同时，减少模型因为过度保守而学不动新任务的问题。

6.5 AFEC: Active Forgetting of Negative Transfer

AFEC 的核心观点是：并非所有旧知识都应该无条件保留。有些旧知识会对新任务产生 negative transfer。AFEC 通过 synaptic expansion-convergence 思路，先扩大模型对新任务的学习能力，再选择性融合有用知识、遗忘会造成负迁移的部分。它的作用是改善 stability 与 plasticity 的平衡。

6.6 ANCL: Auxiliary Networks in Continual Learning

ANCL 引入辅助网络帮助当前任务学习。主网络更偏向稳定保留旧知识，辅助网络更偏向吸收新任务信息。训练时通过正则项在旧模型行为和辅助网络知识之间进行约束与转移。在 BrainWash 的实验中，ANCL 相比部分方法更抗攻击，说明辅助网络机制能在一定程度上缓解 poisoned current-task update 对旧任务表示的破坏。

6.7 ER: Experience Replay

ER 使用 memory buffer 保存部分旧样本：

$$\mathcal{M} = \{(x, y)\}.$$

训练当前任务时，混合当前任务 batch 和 replay batch：

$$\min_{\theta} \mathcal{L}(B_T; \theta) + \mathcal{L}(B_{\mathcal{M}}; \theta).$$

它通过直接复习旧样本降低遗忘。

6.8 ER-ACE: Experience Replay with Asymmetric Cross-Entropy

ER-ACE 针对 class-incremental learning 中 **新旧类别 logits 相互压制的问题**。对当前任务样本, cross-entropy 只在当前任务类别集合 C_T 上计算:

$$\ell_{\text{current}} = -\log \frac{\exp z_y}{\sum_{c \in C_T} \exp z_c}.$$

对 buffer 样本, 则在所有已见类别集合 $C_{1:T}$ 上计算:

$$\ell_{\text{buffer}} = -\log \frac{\exp z_y}{\sum_{c \in C_{1:T}} \exp z_c}.$$

这样 **当前任务学习时不会强行压低旧类别 logits**, 从而减少 abrupt representation change。

7 总结

BrainWash 的关键点是:

- 多 head 指每个任务有独立分类头; 第 t 个 head 的 logits 维度等于该任务类别数;
- MI 不是恢复真实旧数据, 而是 **合成旧任务代理数据**;
- MI 数据用于外层攻击目标, 帮助攻击者估计旧任务遗忘;
- 攻击者真正投毒的是当前任务数据 D_T , 即生成 $x_T^i + \delta^i$;
- 攻击后主要影响模型参数, 而不是直接篡改 buffer;
- 对 regularization-based CL, 攻击通过 **poisoned current-task update** 绕过稳定性正则;
- 对 replay-based CL, 攻击强度与是否能访问 memory buffer 有关, 但论文核心仍是 poisoning current-task data。

参考资料

- Ali Abbasi, Parsa Nooralinejad, Hamed Pirsiavash, Soheil Kolouri. BrainWash: A Poisoning Attack to Forget in Continual Learning. CVPR 2024.
- Hongxu Yin et al. Dreaming to Distill: Data-free Knowledge Transfer via DeepInversion. CVPR 2020.
- Kirkpatrick et al. Overcoming catastrophic forgetting in neural networks. PNAS 2017.
- Aljundi et al. Memory Aware Synapses: Learning what should not be forgotten. ECCV 2018.
- Chaudhry et al. Riemannian Walk for Incremental Learning. ECCV 2018.
- Caccia et al. New Insights on Reducing Abrupt Representation Change in Online Continual Learning. ICLR 2022.